

4팀 발표 질문

프로젝트 공통/개인질문

4팀 공통 질문

1. 발표에서는 L3 스위치 이중화를 구성했다고 설명하셨는데 어떤 이중화 프로토콜을 사용했으며, 실제 장애 상황에서 전환 시간과 패킷 손실은 어느 정도였는지 설명해 주세요.

오명근 답변 :

이번 프로젝트에선 모든 장비가 시스코 사의 장비라고 가정하여 안정성과 벤더 최적화를 위해 HSRP 프로토콜을 사용하였습니다.

HSRP 타이머 설정을 밀리초 단위로 수정하여 장애 발생 시 전환 시간은 1~2초 정도였으며, ping 테스트 결과 0~1개 정도의 패킷 손실이 있었습니다.

2. OWASP CRS를 단계적으로 적용할 계획이라고 말씀하셨는데 현재 구축된 WAF는 공격을 탐지만 하도록 설정한 것인지, 실제 요청 차단까지 수행하도록 설정한 것인지 설명해 주세요. 또한 탐지 모드와 차단 모드를 구분해 운영해야 하는 이유도 함께 말씀해 주세요.

윤철민 답변 :

현재 구축된 WAF Server는 초기 운영 단계의 안정성을 고려하여 탐지 전용 모드로 설정되어 있습니다.

탐지 모드와 차단 모드를 구분해 운영하는 이유는 오탐 방지 때문입니다.

차단 모드로 바로 적용할 경우, 정상적인 사용자 요청까지 공격으로 오인하여 서비스 장애를 유발할 수 있습니다.

따라서 탐지 모드를 통해 충분한 데이터를 수집하고 정책을 튜닝하여 오탐을 최소화한 뒤, 단계적으로 차단 모드로 전환하는 것이 운영의 연속성을 위해 필수적인 단계라고 생각합니다.

3. DingDongDog 서비스의 핵심 데이터가 변조될 수 있는 시나리오를 제시하셨는데 데이터 변조를 방지하기 위해 DB 계정 권한, 애플리케이션 권한 검증, 로그 감사 중 실제로 적용한 통제는 무엇인가요?

서서울 답변 :

저희는 먼저 애플리케이션 연결 계정 권한을 최소 권한 원칙을 적용했습니다. DML 권한만 부여했고 DDL/DCL 권한은 제거했습니다.

그리고 애플리케이션 권한 검증으로 비즈니스 로직 계층에서 세션 정보를 기반으로 요청자의 소유권을 매번 확인해주었습니다.

4. 세션 검증 기능을 적용했다고 하셨는데, 단순히 로그인 여부만 확인한 것인지, 사용자가 요청한 기능이나 데이터에 접근할 권한까지 검증한 것인지 설명해 주세요.

서서울 답변 :

단순히 로그인 여부(인증)만 확인한 것이 아니라, 인가(Authorization) 과정을 포함했습니다.

로그인 시 세션에 저장된 user_id와 요청 데이터의 주체(예: cart_id)를 비교하여, 타인의 데이터에 접근하려는 시도를 차단했습니다.

즉, 요청이 발생할 때마다 "이 세션의 주인이 이 데이터의 접근 권한이 있는가?"를 검증했습니다.

5. WAF 원본 로그를 SIEM에서 분석하기 위해 여러 필드를 추출하셨는데 실제로 추출한 필드와 각 필드를 경로나 분석에 어떻게 활용했는지 설명해 주세요.

윤철민 답변 :

실제 SIEM 분석을 위해 HTTP 요청의 client_ip, request_method, URI, status_code, user_agent 등을 추출하였습니다. client_ip는 비정상적인 접근 근원지 파악 및 동일 IP 기반의 과도한 요청 탐지에 활용하였고, status_code는 에러 패턴을 분석하여 서비스 취약점 공격 시도 여부를 모니터링했습니다. 또한, URI와 user_agent를 통해 특정 경로로 집중되는 공격 징후를 식별하고 비정상적인 Client의 접근 패턴을 분석하는 데 활용했습니다.

6. 정보보안의 기본 요소인 기밀성, 무결성, 가용성을 이번 프로젝트에서는 각각 어떤 기술과 설정으로 확보했는지 구분해서 설명해 주세요.

이주환 답변 :

저희 프로젝트는 정보보안 3대요소를 다음과 같이 확보했습니다 .

저희 프로젝트는 정보보안 3대 요소를 다음과 같이 확보했습니다.

첫째, 기밀성을 위해 IPsec VPN을 통해 통신 구간을 암호화하였습니다.

둘째, 무결성을 위해 Prepared Statement, 화이트리스트 검증, MIME 타입 검증을 적용하여 데이터 변조를 막았습니다.

셋째, 가용성을 위해 L3 스위치 HSRP 이중화를 도입하여, 하나의 장비에 장애가 발생하더라도 네트워크가 끊기지 않도록 유지했습니다.

7. 전체 구성에서 WAF, UTM, SIEM을 연계하셨는데 실제 운영 중 장애가 발생한다면 가장 우선적으로 확인해야 할 단일 장애 지점은 어디라고 판단하셨습니까?

서정원 답변 :

저희 구성에서 가장 우선적으로 확인해야 할 단일 장애 지점은 UTM이라고 판단했습니다.

UTM이 외부와 내부 네트워크 사이의 앞단에 위치하고 있으며, 대부분의 서비스 트래픽이 UTM을 거쳐 WAF와 웹 서버로 전달되기 때문입니다. 따라서 UTM에 장애가 발생하거나 정책이 잘못 적용되면 WAF와 웹 서버가 정상적으로 동작하더라도 전체 서비스 접근이 차단될 수 있습니다.

장애 발생 시에는 먼저 UTM 장비의 인터페이스 및 서비스 상태, 라우팅과 방화벽 정책의 허용·차단 여부를 확인할 것입니다. 동시에 SIEM에서 UTM과 WAF 로그가 정상적으로 수집되고 있는지도 확인하여, 실제 장비 장애인지 정책에 의한 차단인지 빠르게 구분하겠습니다.

따라서 서비스 가용성 측면에서는 UTM을 최우선으로 확인하고, 원인 분석과 보안 관제 측면에서는 SIEM을 함께 확인하는 방식으로 대응하겠습니다.

8. Docker 환경에서 Web과 DB를 동일한 EC2 인스턴스에 구성하셨는데 해당 EC2에 장애가 발생하면 두 서비스가 동시에 중단될 수 있습니다. 이러한 구조를 선택한 이유와 실제 운영 환경에서의 개선 방안을 설명해 주세요.

서서울 답변 :

저희는 개발과 테스트 단계에서 개발자가 의도한 환경을 배포 시에도 그대로 재현하여, 안정적이고 신뢰할 수 있는 서비스를 운영하기 위해서 이러한 구조를 선택했었는데 질문 주신대로 만약 EC2 장애가 발생한다면 한번에 중단될 수 있다는 단점이 존재합니다. 이를 해결하기 위해서는 첫번째로 DB를 AWS RDS로 이관해 인스턴스 하나가 중단되어도 자동으로 장애조치가 일어나 서비스 중단을 방지 할 수 있습니다.

두번째로는 Application Load Balancer를 사용해 사용자의 요청을 받고 Auto Scaling을 사용해 트래픽을 분산해 주는 방법도 있습니다.

9. 발표 중 TCP 로그 전송을 통해 무결성을 보장했다고 설명하셨는데 TCP는 전송 신뢰성과 순서는 보장하지만 공격자에 의한 로그 위·변조까지 막지는 못합니다. 실제 로그 무결성을 확보하려면 어떤 추가 통제가 필요하다고 생각하십니까?

윤철민 답변 :

TCP는 전송의 신뢰성과 순서만을 보장할 뿐, 전송 과정에서의 데이터 위·변조를 막는 보안 프로토콜은 아닙니다.

실제 로그 무결성을 확보하기 위해서는 세 가지 기술적 통제가 추가로 필요하다고 생각합니다.

첫째, 전송 구간에 TLS 적용을 통해 통신 경로 자체를 암호화함으로써 중간자 공격을 방지하고 전송 중 데이터 변조를 차단할 수 있습니다. 둘째, 데이터 자체에 대한 인증이 필요합니다. 로그를 생성할 때 HMAC이나 디지털 서명을 포함하여, 수신 측에서 로그 수신

시 무결성 검증을 수행하도록 설계해야 합니다. 셋째, WORM Storage 활용입니다. 수신된 로그를 한 번 기록하면 삭제나 수정할 수 없는 저장소에 보관함으로써, 저장 이후의 로그 변조 가능성까지 원천적으로 차단하는 통제가 있어야 한다고 생각합니다.

서서울

1. 전체 프로젝트 기간이 31일이었다고 하셨는데, 제한된 기간 중 가장 우선순위를 높게 설정한 작업은 무엇이었으며, 해당 작업을 먼저 수행한 이유는 무엇인가요?

WBS의 초기 단계인 '취약점 점검대상 식별 및 분류'부터 '취약점 본 점검 수행'까지(6월 9일 ~ 6월 25일)에 해당하는 전체 분석 작업을 최우선으로 완료하는 것이었습니다. 단순히 일정이 앞에 있어서가 아니라, 보안 프로젝트의 성공은 정확한 위험 식별에서 시작된다는 생각 때문이었습니다. 구체적인 이유는 다음과 같습니다.

첫번째로 보안 리소스의 효율적 배분입니다. 전체 인프라 중 어떤 자산(서버, 네트워크, 애플리케이션)이 가장 취약하고, 어떤 공격 시나리오(SQL Injection, IDOR 등)에 노출되어 있는지 초기에 정밀하게 타격 지점을 파악해야 했습니다. 이를 먼저 완료해야 이후 이어지는 '인프라 구축'과 '보안 설정' 단계에서 리소스를 낭비하지 않고 가장 위험한 곳에 우선적으로 보안 통제를 적용할 수 있기 때문입니다.

두번째로 재작업(Rework) 방지입니다. 만약 인프라를 먼저 구성하고 나중에 취약점을 발견했다면, 이미 구축된 시스템을 뜯어고치는 큰 비용이 발생했을 것입니다. 분석을 최우선으로 진행하여 설계 단계부터 보안 요소(망 분리, 파라미터 바인딩 등)를 반영함으로써, 프로젝트 후반부의 수정 작업을 최소화하여 31일이라는 타이트한 일정을 완수할 수 있었습니다.

2. 프로젝트에서 발견하거나 재현한 취약점 중 위험도가 가장 높았던 취약점은 무엇인가요? 발생 가능성과 영향도를 어떤 기준으로 판단했는지 설명해 주세요.

가장 위험도가 높은 취약점은 IDOR입니다.

먼저 발생 가능성으로 유기견 성향 데이터를 기반으로 입양가족을 매칭하는 서비스로 회원 및 유기견 데이터는 매우 민감한 자산입니다. 이를 간단하게 인가단계의 누락으로 다른 회원들이 정보를 탈취하거나 변조가 가능해지는건 서비스의 신뢰성 자체를 무너트릴 수 있는 위험한 취약점이라 판단했습니다.

영향도로는 매칭 서비스에서 가장 중요한 것은 데이터의 무결성(Integrity)입니다. 비인가자가 IDOR을 통해 타인의 개인정보를 열람하거나, 매칭 데이터를 임의로 변조할 경우 서비스의 신뢰성은 회복 불가능한 타격을 입습니다.

IDOR은 서비스의 근간인 매칭 알고리즘과 데이터의 신뢰도를 파괴하는 운영/비즈니스 보안의 문제라고 판단했습니다.

3. 사업 목표로 보안 위협의 신속한 탐지와 대응을 제시하셨는데 현재 프로젝트에서는 탐지 이후의 대응 절차를 어디까지 실제로 구현했습니까? 경보 확인, 분석, 차단, 보고 중 구현한 범위를 설명해 주세요.

보안 위협 탐지 이후 저희가 수행한 대응 절차는 '경보 확인(필터링) → 오탐 여부 판단 → 분석 및 차단 → 사후 조치(보고)' 순으로 진행됩니다. 탐지 이후의 상세 대응 과정은

다음과 같습니다.

1. 신속한 탐지 및 오탐 확인(필터링)

우선 탐지된 경보가 실제 공격인지, 아니면 정상적인 요청이 보안 정책에 걸린 '오탐 (False Positive)'인지 판단하는 과정을 최우선으로 합니다.

- 오탐 판단 기준: 저희 프로젝트에서는 애플리케이션 로그와 보안 장비의 로그를 교차 검증합니다. 예를 들어, 사용자가 정상적인 매칭 로직을 이용하는 도중, 입력 값에 특수문자가 포함되어 보안 정책에 의해 탐지된 경우, 해당 요청의 사용자의 의도와 요청의 정당성을 분석합니다.
- 구현 방법: 해당 요청이 '사용자가 서비스를 정상적으로 이용하는 과정에서 발생하는 데이터'인지, 아니면 '시스템의 허점을 노리고 의도적으로 조작된 데이터'인지 로그를 통해 확인합니다. 이 과정에서 정상적인 업무 처리 과정의 일환으로 확인될 경우, 탐지 정책을 화이트리스트로 조정하여 서비스 운영의 안정성을 보장합니다.

2. 분석 및 차단

오탐이 아닌 실제 공격으로 판단될 경우, 다음과 같이 대응을 구현했습니다.

- 분석: 탐지된 공격 패턴을 분석하여 공격자가 노린 데이터와 취약점 지점(예: 장바구니 파라미터 변조 등)을 식별합니다.
- 차단:
 - 즉각적인 대응: 현재 프로젝트 환경에서는 WAF(mod_security)를 통해 탐지된 SQL Injection 공격 패턴을 실시간 차단하도록 설정했습니다.
 - 애플리케이션 계층 대응: IDOR과 같은 논리적 취약점은 WAF 설정만으로는 한계가 있습니다. 이를 위해 해당 요청이 발생한 소스 코드의 로직을 수정하여, 세션 정보와 데이터 소유주가 일치하지 않는 요청은 서버단에서 403 Forbidden 응답을 반환하도록 강제 차단했습니다.

3. 보고 및 운영 체계

- 탐지된 모든 보안 이벤트와 차단 결과는 로그 파일에 기록됩니다.
- 최종 조치: 프로젝트 종료 시점에 작성한 결과 보고서에는 단순히 '몇 건의 공격이 있었다'는 수치뿐만 아니라, '어떤 경로로 침입 시도가 있었고, 현재 어떤 보안 정책을 통해 이를 원천 차단하고 있는지'를 정리하여 운영자가 향후 동일한 공격 유형에 대해 즉시 대응할 수 있도록 가이드를 마련했습니다.

4. SQL Injection 검증 과정에서 SUBSTRING(database(), 1, 1) = 'd' 조건을 사용하셨는데 해당 조건이 참인지 거짓인지는 애플리케이션의 어떤 응답 차이를 통해 판단했는지 설명해 주세요.

저는 로그인 페이지의 응답 상태(로그인 성공 여부)를 지표로 활용했습니다.

참일 때는 조건식이 참이 되어 전체 쿼리문이 '정상적인 로그인'으로 인식됩니다. 이 경우, 애플리케이션은 로그인 성공 알림(예: "환영합니다!")을 띄우거나, 메인 페이지(main.php)로 리다이렉트(Redirect)하는 정상적인 응답을 반환합니다.

거짓일 때는 조건식이 거짓이 되어 쿼리문의 논리적 결과가 부정됩니다. 이 경우, 애플리케이션은 "정보가 일치하지 않습니다"와 같은 로그인 실패 메시지를 띄우거나, 에러 페이지를 반환하는 등의 차이를 보입니다.

5. 장바구니 요청에서 cart_id와 cart_quantity 값이 평문으로 확인됐다고 설명하셨는데 이러한 값이 평문으로 보인다는 사실 자체가 취약점이라고 볼 수 있을까요? 실제 취약점으로 판단하려면 어떤 추가 조건이 필요합니까?

평문으로 보이는 것 자체는 바로 취약점이라고 보기는 어렵습니다. 실제 취약점으로 판단하기 위해서는 서버 측의 인가 검증 로직 부재가 있어야 합니다.

첫번째로 권한 확인 로직의 부재로 공격자가 패킷을 가로채서 cart_id를 타인의 id로 바꾸거나, cart_quantity를 비정상적으로 높게 조작해서 전송했을 때, 서버가 해당 요청을 보낸 사용자가 진짜 그 장바구니의 소유주인지 확인하고 그대로 로직을 실행하는 경우가 있습니다.

두번째로 인가 실패로 서버가 세션의 user_id와 요청받은 cart_id의 소유권 정보를 대조하여 인가되지 않은 접근으로 판단하고 차단할 수 있어야 하는데, 이 과정이 빠져있을 때 평문 노출이 IDOR 취약점이라는 공격으로 이어질 수 있습니다.

6. Docker 컨테이너와 VM은 모두 서비스를 격리해 실행할 수 있는 환경입니다. 두 기술의 구조적인 차이와 자원 사용, 격리 수준, 운영 방식의 차이를 설명해 주세요.

1. 구조적 차이 (Architecture)

- VM (Virtual Machine): 하드웨어를 가상화합니다. 하이퍼바이저(Hypervisor)라는 소프트웨어가 하드웨어 자원을 쪼개고, 그 위에 각각 독립적인 Guest OS(커널 포함)를 실행합니다. 즉, 물리 서버 위에 여러 개의 OS가 통째로 올라가는 구조입니다.
- Docker 컨테이너: OS를 가상화합니다. 호스트 OS의 커널을 공유하며, 컨테이너 엔진(Docker Engine)이 프로세스를 격리합니다. 별도의 Guest OS 없이 애플리케이션과 실행에 필요한 라이브러리/바이너리만 패키징하여 실행합니다.

2. 자원 사용 (Resource Efficiency)

- VM: 각 VM마다 독립된 OS가 동작하므로 메모리와 CPU 자원을 상당 부분 OS 자체가 점유합니다. 부팅 속도가 느리고, 무거운 하드웨어 에뮬레이션 작업이 필요합니다.

- Docker 컨테이너: OS를 공유하므로 불필요한 자원 낭비가 거의 없습니다. 컨테이너는 실행 중인 프로세스에 불과하므로, 부팅 속도가 밀리초(ms) 단위로 매우 빠르고 매우 가볍습니다.

3. 격리 수준 (Isolation)

- VM: 하드웨어 수준에서 격리되므로 격리 수준이 매우 높고 보안상 강력합니다. 한 VM이 해킹당해도 다른 VM이나 호스트로 영향을 주기 어렵습니다.
- Docker 컨테이너: 프로세스 수준의 논리적 격리입니다. 커널을 공유하기 때문에 VM보다는 격리 강도가 상대적으로 낮습니다. 하지만 최근에는 보안 강화를 위한 네임스페이스(Namespace)와 Cgroups 기술이 고도화되어 운영 환경에서 충분한 격리 수준을 제공합니다.

4. 운영 방식 (Operation)

- VM: 인프라 설정, 업데이트, 패치 등 OS 관리가 필요합니다. 이식성은 좋지만, 이미지를 배포하기에는 용량이 크고 무겁습니다.
- Docker 컨테이너: 이식성(Portability)이 극대화됩니다. "내 환경에서는 돌아가는데 서버에서는 왜 안 되지?"라는 문제를 해결하기 위해, 개발 환경과 동일한 런타임 환경을 이미지로 만들어 어디든 똑같이 배포할 수 있습니다.

오명근

1. 본인이 생각하는 네트워크 엔지니어의 역할은 무엇인가요? 단순한 장비 설정 외에 실제 운영 환경에서 담당해야 하는 업무까지 포함해 설명해 주세요.

네트워크 엔지니어는 트래픽 흐름을 예상하여 인프라의 뼈대를 설계하고 실제로 구축하는 것은 물론, 무단 접근 방지를 위한 보안 정책을 적용하기도 합니다. 또한 지연 시간이나 패킷 손실 등을 모니터링하며 트래픽 경로를 최적화하고 장애 발생 시 원인을 파악하고 신속히 대응하기도 합니다.

결론적으로 네트워크 엔지니어는 단순히 장비 설정만 하는게 아니라 서비스가 문제없이 안정적으로 운영되도록 하기 위해 인프라를 전체적으로 설계하고 관리하는 역할을 가지고 있다고 생각합니다.

2. L3 스위치 이중화 방식으로 HSRP를 선택하셨는데 HSRP를 선택한 이유와 VRRP와의 주요 차이를 설명해 주세요. 실제 환경에서는 어떤 기준으로 두 프로토콜 중 하나를 선택할 수 있을까요?

이번 프로젝트 진행시에 모든 장비를 시스코 사의 장비라고 가정했기 때문에 벤더 최적화를 위해 HSRP 프로토콜을 사용하기로 했습니다.

HSRP와 VRRP는 유사한 점이 많지만 몇가지 차이점이 있습니다. 제일 큰 차이점은 호환성으로, HSRP는 시스코 독점 프로토콜이지만 VRRP는 멀티 벤더에서 호환이

가능한 프로토콜입니다. 또한 장비 역할을 active / standby와 master / backup으로 구분한다는 차이점도 있습니다.

실제 환경에서는 멀티 벤더 환경인가의 여부로 두 프로토콜을 선택하게 됩니다. 인프라를 전부 시스코 장비로 구성하게 된다면 HSRP 프로토콜을 선택하는게 유리하고, 여러 제조사의 장비를 사용하고 있거나 추후 교체, 확장하며 다른 제조사의 장비를 사용할 거라면 호환성이 보장되는 VRRP 프로토콜을 선택하는게 유리하다고 생각합니다.

3. 본사와 지사 간 연결에 GRE over IPsec을 적용하셨는데 GRE만 사용했을 때와 GRE over IPsec을 사용했을 때의 차이를 기밀성 관점에서 설명해 주세요.

GRE는 자체적인 보안 설정이 존재하지 않기 때문에 공격자가 중간에서 패킷을 탈취한다면 사실 ip 정보와 내부 데이터 전체를 평문으로 볼 수 있어 기밀성이 없습니다.

반면 GRE over IPsec은 GRE 가상 터널 위에 보안 프레임워크인 IPsec을 더해 데이터를 한번 캡슐화하고, 암호화 알고리즘을 적용하여 데이터의 기밀성을 확보한다는 차이가 있습니다.

4. 형상관리 과정에서 파일명에 작성자와 날짜를 추가해 버전 혼선을 줄였다고 하셨는데, 두 명이 동일한 파일을 동시에 수정하는 경우에는 해당 방식만으로 충돌을 방지하기 어려울 것 같습니다. 실제 협업 환경에서는 어떻게 관리할 수 있을까요?

이번 프로젝트 진행시 문제가 생겼던 산출물들은 엑셀이나 워드, ppt같은 파일들이었기 때문에 두 명이 동일한 파일을 동시에 수정한 경우에는 취합시에 검토만 잘 하면 충돌은 일어나지 않았습니다. 다만 한 사람이 하루에 여러번 수정을 하며 계속하여 공유했을 경우에는 어떤것이 최신 파일인지 혼동이 오는 문제가 있었습니다.

실제 협업 환경에서는 규모가 크기 때문에 더욱 체계적인 형상관리가 필요하다고 생각합니다. 만약 분산 작업 후 병합이 가능한 파일이라면 git과 같은 분산 버전 관리 시스템을 사용하여 형상관리를 하는것이 좋다고 생각합니다. 또한 일반 문서라면 클라우드 기반 공동 문서 편집 도구를 사용하여 하나의 파일을 같이 작업하며 버전을 관리하는것이 적절하다고 생각합니다.

5. HSRP Active 장비 자체의 장애 전환은 검증하셨는데 장비는 정상이고 외부로 연결되는 업링크 인터페이스만 장애가 발생하는 경우에도 전환될 수 있도록 Interface Tracking을 구성하셨나요? 구성하지 않았다면 어떤 문제가 발생할 수 있습니까?

업링크 인터페이스 장애 발생 시에도 active 장비가 전환될 수 있도록 인터페이스 트래킹 설정도 적용하였습니다.

이 설정이 없을 시 업링크에서 장애가 발생해도 장비 자체는 정상이기 때문에 전환이 일어나지 않습니다. 트래픽들은 현재 활성화된 active 장비로 전송되지만 상위 링크에서 장애가 발생했기 때문에 전부 드랍되며 통신이 마비되는 문제가 발생합니다.

윤철민

1. WAF를 웹 서버에 직접 설치하지 않고 Reverse Proxy 방식으로 구성하셨는데 해당 방식을 선택한 이유와 보안 및 운영 측면에서의 장점을 설명해 주세요.

WAF를 WEB Server에 직접 설치하지 않고 Reverse Proxy 방식으로 구성한 이유는 보안 격리와 운영 효율성 때문입니다. 보안 측면에서는 WEB Server의 실제 IP를 외부에 노출하지 않아 직접적인 공격을 방어할 수 있고, 웹 서버와 분리되어 있어 WAF가 공격받더라도 백엔드 서버로의 침투를 예방할 수 있습니다. 또한, 웹 서버 앞단에 배치하여 중앙 집중적인 정책 관리가 가능하다는 장점이 있습니다.

2. rsyslog에서 imfile 모듈을 사용하셨는데 imfile 모듈이 어떤 역할을 하며, 일반적인 syslog 메시지 수신 방식과 어떤 차이가 있는지 설명해 주세요.

일반적인 syslog 방식이 Network(UDP/TCP)를 통해 전송되는 로그 메시지를 수신하는 것에 반면, imfile

모듈은 Server 내 특정 경로에 생성되는 로그 파일의 변화를 실시간으로 추적하여 파일 내용을 읽어 syslog 시스템으로 전달합니다. Network Socket뿐만 아니라 로컬 파일 단위의 로그까지 수집 범위를 확장하는 핵심 모듈입니다.

3. DNSSEC를 적용하고 Zone Signing을 수행했다고 하셨는데 서명이 정상적으로 적용됐다는 것을 어떤 DNS 레코드와 명령어로 검증했는지 설명해 주세요.

DNSSEC 검증은 운영체제별로 도구를 달리하여, 두 환경 모두에서 무결성 체계가 정상 작동함을 확인했습니다.

첫째, Linux 환경에서는 "dig +dnssec [domain_name] [record_type]" 명령을 실행하여 RRSIG 레코드 포함 여부를 확인했습니다. 또한 "dig DNSKEY [domain_name]" 명령으로 공개 키와 DS Record를 확인하여 상위 도메인과의 신뢰 연결을 검증했습니다.

둘째, Windows Server 환경에서는 PowerShell의 "Resolve-DnsName -Name [domain_name] -Server 127.0.0.1 -Type A -DnssecOk" 명령과 delv 명령어를 활용하여 서명된 응답을 확인했습니다. 결과적으로 Linux와 Windows 환경 모두에서 DNS Spoofing 공격에 대응할 수 있는 무결성 보안 체계가 실질적으로 동작하고 있음을 교차 검증했습니다.

이주환

1. CSRF와 XSS는 모두 웹 애플리케이션에서 발생하는 공격이지만 공격 원리에는 차이가 있습니다. 두 공격의 발생 원인과 공격 대상, 피해 방식의 차이를 설명해 주세요.

먼저 XSS는 사용자 입력값을 출력할 때 HTML 인코딩을 하지 않아 브라우저에서 악성 JavaScript가 실행되는 취약점입니다. 공격 대상은 사용자의 브라우저이며, 공격자가 삽입한 JavaScript가 피해자의 브라우저에서 실행됩니다. 이로 인해 로그인된 상태에

서는 쿠키 탈취나 세션 하이재킹이 가능하고, 로그인 여부와 관계없이 피싱 화면 생성이나 악성 사이트 리다이렉트 등의 피해가 발생할 수 있습니다.

반면 CSRF는 브라우저의 자동 쿠키 전송 특성과 서버가 CSRF Token이나 Origin 검증 등 요청의 정당성을 검증하지 않을 때 발생하는 취약점입니다. 공격 대상은 웹 서버입니다. 서버가 브라우저에서 자동 전송된 세션 쿠키만으로 요청을 정상 사용자의 요청이라고 잘못 판단하여 상태 변경 요청을 처리하기 때문입니다. 이로 인해 사용자의 개인정보 변경, 비밀번호 변경, 회원 탈퇴 등의 피해가 발생할 수 있습니다.

2. 파일 업로드 시 난수 형태로 파일명을 변경했다고 하셨는데 공격자가 업로드 응답이나 페이지 소스를 통해 실제 저장된 파일명을 확인한다면 다시 접근할 수도 있지 않을까요? 난수 파일명 외에 어떤 추가 통제가 필요합니까?

맞습니다.

난수 파일명은 파일명을 예측하기 어렵게 만드는 보조적인 보안 대책일 뿐입니다.

따라서 저장된 파일명이 노출될 경우 다시 접근이 가능하기 때문에 확장자를 검증하는 화이트리스트방식을 통해 제한하고, 확장자 검증만으로는 충분하지 않기 때문에 MIME 타입과 파일내용을 직접 검사하여 php실행 파일이 업로드 되지 않도록 해야합니다.

혹은 Apache 에서 .htaccess 또는 서버 설정으로 php 엔진 실행을 막아 파일이 업로드 되더라도 실행되지 않도록 할 수 있습니다.